

xbin

Ship your app like a binary. Run anywhere.

Technical Documentation — Secure by Design — v0.1 — 2026



Rust

Python

Ed25519

Linux Namespaces

seccomp

zstd

1. Le problème

Distribuer une application logicielle est inutilement compliqué. Une app dépend d'un runtime (Node.js, Python, Java...), de bibliothèques dynamiques système (fichiers `.so`), de packages (node_modules, pip...), et d'assets. Sur une autre machine, **ça casse**. C'est le problème historique du "ça marche chez moi".

Docker a partiellement résolu ça mais impose une friction énorme : daemon root, images, registries, networking. Trop lourd pour distribuer un simple outil CLI ou une app interne.

xbin résout ça en une commande

- `chmod +x mon_app && ./mon_app`
- Zéro installation. Zéro dépendance. Zéro configuration.
- Un seul fichier binaire qui contient absolument tout.

2. Secure by Design

Un outil qui distribue et exécute du code arbitraire est une cible naturelle. Si xbin n'est pas sécurisé par conception, il devient un vecteur d'attaque parfait. La sécurité n'est pas ajoutée après — elle est dans chaque décision d'architecture.

2.1 Les 4 failles du design naïf

Faillle 1 — Absence d'authenticité

- Problème : n'importe qui peut créer un `.xbin`. Aucune vérification de la source.
- Solution : signature Ed25519 obligatoire. Le launcher vérifie avant d'extraire.
- Si signature invalide ou absente → refus immédiat, rien écrit sur le disque.

- Problème : entre le check du hash et l'utilisation, un attaquant injecte du contenu.
- Solution : extraction atomique via `rename()` — opération atomique sur Linux.
- Rien n'est mis en cache tant que la vérification complète n'est pas passée.

Faillle 3 — Fallback `LD_LIBRARY_PATH` non sécurisé

- Problème : l'app voit le filesystem host, vulnérable aux fausses bibliothèques.
- Solution : pas de fallback silencieux. Si namespaces indisponibles → refus + explication.
- Mieux refuser proprement que tourner en mode non sécurisé silencieusement.

Faillle 4 — Hash dans le fichier lui-même

- Problème : un attaquant qui modifie le payload peut mettre à jour le hash.
- Solution : signature cryptographique Ed25519 sur SHA-256(payload + metadata).

- Sans la clé privée, impossible de forger une signature valide.

2.2 Le schéma secure by design

Voici la séquence complète d'exécution sécurisée. Chaque étape doit passer avant que la suivante commence. Un échec à n'importe quelle étape arrête tout proprement.

Secure execution flow

Every step must pass before the next begins



☐ Cryptographic verification

☐ Kernel isolation

☐ App running

Règle fondamentale

- Rien n'est écrit sur le disque avant que la signature Ed25519 soit vérifiée.

3. Le format binaire .xbin

Un fichier .xbin est un exécutable ELF valide avec des données supplémentaires collées après le code. Le kernel Linux charge et exécute la partie ELF. Le launcher, une fois démarré, s'ouvre lui-même via /proc/self/exe pour lire le payload et les métadonnées.

Magic	Version	Payload	SHA-256	Signature	Pub key	ELF	payload	metadata	trailer
XBIN\x01	Arch Flag	offset+size	32 bytes	Ed25519 64B	32 bytes	launcher	zstd	JSON	magic

.xbin binary file format — bytes left to right

Offset 0-4	: Magic "XBIN\x01"	(5 bytes)	identifiant
Offset 5	: Version (u8)	(1 byte)	versioning
Offset 6	: Architecture (u8)	(1 byte)	x86_64=0x01
Offset 7	: Flags (u8)	(1 byte)	signé/chiffré
Offset 8-15	: Payload offset (u64 LE)	(8 bytes)	localisation
Offset 32-63	: SHA-256 du payload	(32 bytes)	intégrité
Offset 80-143	: Signature Ed25519	(64 bytes)	authenticité
Offset 144-175	: Clé publique Ed25519	(32 bytes)	vérification
[ELF code du launcher]			
[payload zstd compressé]			
[metadata JSON]			
[trailer: offset(u32) + magic 0xBEEFCAFE(u32)]			

4. L'isolation via Linux namespaces

Linux propose 8 types de namespaces. xbin en utilise 3 pour créer un environnement isolé sans aucun privilège root.

Namespace	Flag	Ce que ça isole	Pourquoi xbin l'utilise
User	CLONE_NEWUSER	UIDs et GIDs	Permet d'être "root" dans le namespace sans l'être sur le host
Mount	CLONE_NEWNS	Points de montage	pivot_root pour changer la racine du filesystem
PID	CLONE_NEWPID	IDs de processus	L'app ne voit pas les processus du host

```
# Séquence d'isolation (Rust, sans root)
unshare(CLONE_NEWUSER | CLONE_NEWNS | CLONE_NEWPID)?
write("/proc/self/uid_map", "0 1000 1\n")? // root dans ns = uid 1000 dehors
write("/proc/self/setgroups", "deny")? // obligatoire
write("/proc/self/gid_map", "0 1000 1\n")?
mount(rootfs, rootfs, MS_BIND | MS_REC)? // bind-mount
pivot_root(rootfs, rootfs + "/old_root")?
umount2("/old_root", MNT_DETACH)? // host filesystem DISPARAIT
```

```
apply_seccomp_filter()?           // syscalls filtrés  
execve(entrypoint, args, env)?   // app démarre
```

5. La stack technique

Composant	Technologie	Raison
Launcher stub	Rust + musl statique	Sécurité mémoire, zéro dépendance, < 200KB
Cryptographie	ed25519-dalek (Rust)	Auditée, standard moderne, résistante
Compression	zstd niveau 3	Ratio/vitesse optimal, décompression rapide
Namespaces	nix crate (Rust)	Wrapper safe sur les syscalls Linux
Seccomp	seccompiler (Rust)	Filtrage syscalls, appliqué avant exec
CLI / Builder	Python 3 + uv	Logique métier, développement rapide
Analyse IA	Claude API (anthropic SDK)	Détection des dépendances cachées
Package manager	uv	10-100x plus rapide que pip

5.1 Pourquoi Rust pour le launcher ?

Le launcher est le composant le plus critique. Il manipule des fichiers binaires, fait des appels système bas niveau, et s'exécute avant toute vérification. Une erreur mémoire ici = faille de sécurité.

- Le borrow checker élimine use-after-free et buffer overflows par construction
- Compilation statique avec musl → zéro dépendance dynamique → tourne partout
- ed25519-dalek, sha2, zstd, nix, seccompiler — des crates auditées et matures
- opt-level = "z" + lto + strip → binaire < 200KB strippé

5.2 Pourquoi Python pour le builder ?

Le builder est de la logique métier : parcourir des répertoires, appeler ldd, détecter des patterns, appeler une API. Python est optimal pour ça.

- Bibliothèque standard riche : pathlib, subprocess, hashlib, json
- uv : gestionnaire de paquets 10-100x plus rapide que pip
- SDK anthropic disponible → intégration Claude API triviale
- Rapide à itérer et modifier pendant la phase de développement

5.3 Le rôle précis de l'IA

L'IA résout un seul problème — mais il est réel : la détection des dépendances que ldd ne voit pas.

```
# Ce que ldd ne détecte PAS :
subprocess.run(["ffmpeg", "-i", input, output]) # binaire externe
ctypes.cdll.LoadLibrary("libcuda.so.1")        # dlopen dynamique
os.system("convert image.png output.jpg")       # ImageMagick

# xbin AI analyse le code et génère :
External binaries: ffmpeg, convert
```

Dynamic libs: libcuda.so.1 (optional, GPU only)
Env vars required: DATABASE_URL, SECRET_KEY

6. Comparaison avec les alternatives

Critère	xbin	ApplImage	Docker	Nix
Zéro installation	✓	✓	✗	✗
Détection auto des deps	✓ + IA	✗ manuel	✗	✗
Cache intelligent	✓	✗	✓	✓
Isolation sans root	✓	✗	✗	✓
Signature obligatoire	✓	optionnel	✗	✓
Warm start < 100ms	✓	✓	✗	✗
IA pour deps cachées	✓	✗	✗	✗

7. Ordre d'implémentation

Semaine 1-2 — Format et vérification cryptographique

- Spécifier le format .xbin sur papier — chaque field, chaque offset
- verify.rs : génération clés Ed25519, signature, vérification
- format.rs : parsing du header binaire
- Tests unitaires exhaustifs incluant les cas d'attaque

Semaine 3 — Launcher stub complet

- Self-reading via /proc/self/exe et localisation du payload
- Cache atomique avec flock() et LRU cleanup
- Décompression zstd du payload
- Tests sur Ubuntu 18.04, 20.04, 22.04 en container vierge

Semaine 4 — Isolation Linux

- User namespaces + uid/gid mapping
- pivot_root + démontage de l'ancien root
- Filtre seccomp minimal
- Vérification : l'app ne peut pas lire /etc/passwd du host

Semaine 5 — Builder Python

- Détection des dépendances via ldd récursif
- Détection du runtime (Python / Node / Bash / ELF natif)
- Construction du rootfs et compression zstd
- Assemblage du fichier .xbin final avec signature

Semaine 6+ — CLI et IA

- CLI complète : build, sign, run, inspect, clean
- Intégration Claude API pour l'analyse de code
- Génération automatique du xbin.manifest
- Performance : warm start < 100ms mesuré avec hyperfine

8. Métriques cibles

Métrique	Cible
Warm start (cache hit)	< 100ms
Cold start (première extraction)	< 5s pour app Python type
Taille launcher stub strippé	< 200KB
App Python 50MB non compressée	→ ~14MB compressée (ratio ~3.5x)
Overhead namespaces Linux	< 20ms
Overhead vérification signature	< 1ms (Ed25519 est rapide)

Ce que xbin n'est pas

- Pas un Docker killer — Docker reste pour l'orchestration et le cloud
- Pas une VM — l'app tourne sur le kernel host, pas un kernel virtualisé
- Pas un gestionnaire de paquets — pas de registry central
- Positionnement juste : "pour les cas où Docker est trop lourd"